

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

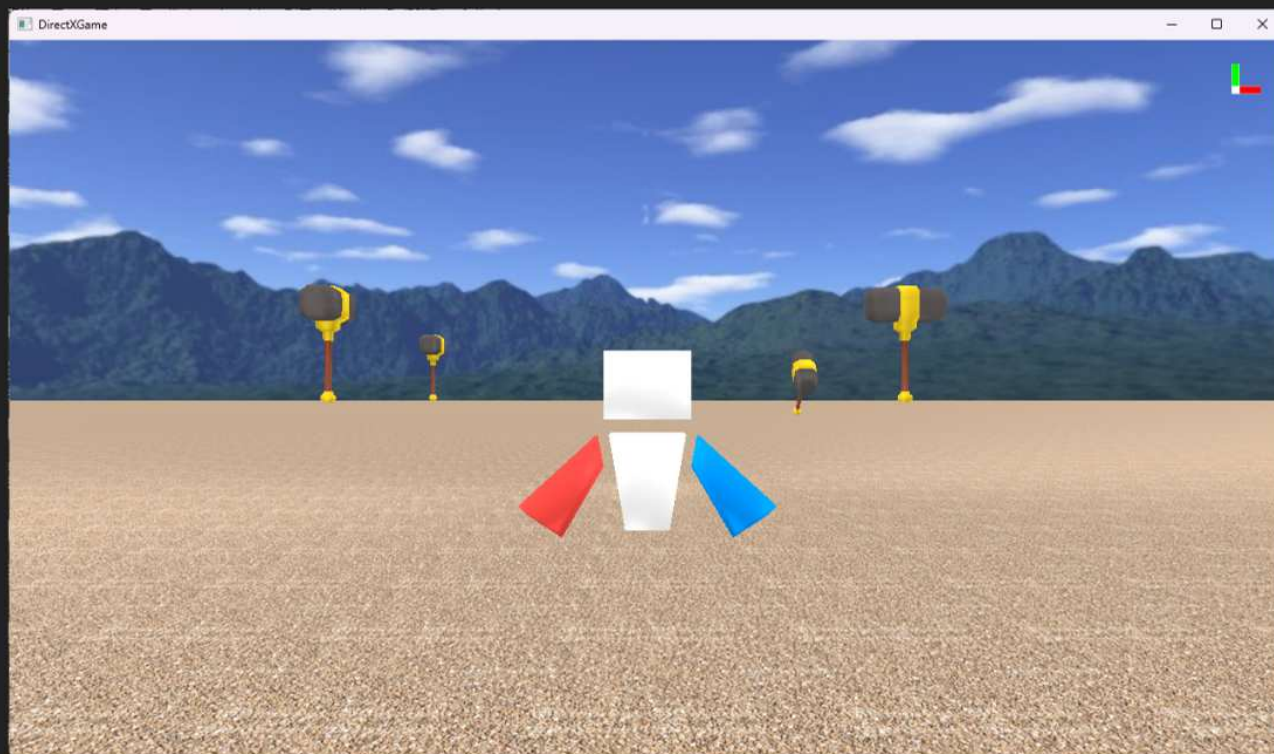
02_04.自キャラSpawnPoint

日本工学院専門学校
デザインカレッジ

実行結果

docs.google.com – 全画面表示を終了するには、Esc を押します

日本工学院専門学校
デザインカレッジ



Blenderで設定したSpawnPoint上に
自キャラが配置されるようになった。

Imguiで座標情報を表示したり、
背景オブジェクトとの位置関係を見
て、配置が正しいかどうかを確認
しておこう。

自キャラに座標を反映

日本工学院専門学校
デザインカレッジ

ゲームプレイシーンなど

```
// 自キャラの生成
自キャラの生成
// 自キャラの初期化
自キャラの初期化

// プレイヤー配置データからプレイヤーを配置
if (!levelData->players.empty()){
    auto& playerData = levelData->players[0];
    player->座標設定(playerData.translation);
    player->回転角設定(playerData.rotation);
}
```

読み込んだ配置データを自キャラに反映する処理。

Jsonファイルに自キャラの配置データがない場合も想定して、生成と配置は処理を分離した方が無難。

自陣営のキャラが複数いるタイプのゲームであればfor文で生成と配置を配置データの数だけ回すとよいだろう。

オブジェクトの走査

日本工学院専門学校
デザインカレッジ

レベルデータローダー

```
// "objects"の全オブジェクトを走査
for (...省略...) {

    ...省略...

    // MESH
    if (type.compare("MESH") == 0) {
        ...省略...
    }
    // 自キャラ発生ポイント
    else if (type.compare("PlayerSpawn") == 0) {
        playersに要素を1つ追加する
        平行移動の数値を書き込む
        回転角の数値をラジアンで書き込む
    }
}
```

Jsonファイルのデータを走査して記録する処理が既にあるが、そこに"**type**" 欄が "**PlayerSpawn**" の場合の処理を追加する。

といっても**PlayerSpawnData**に特別なデータはないので、背景オブジェクトについての処理とほとんど変わらない。

独自のデータを追加した場合は、ここで吸収することになる。

データ構造体

日本工学院専門学校
デザインカレッジ

```
// 自キャラの生成データ
struct PlayerSpawnData {
    // 平行移動
    KamataEngine::Vector3 translation;
    // 回転角
    KamataEngine::Vector3 rotation;
};
```

レベルデータローダー

いよいよ、書き出したレベルデータから実際にゲームに配置する処理を書いていこう。

テキストでは KAMATA ENGINE の例でポイントだけ紹介していく。

自作エンジンの場合はうまく吸収してほしい。

```
// レベルデータ
struct LevelData {
    ...省略...

    // 自キャラ配列
    std::vector<PlayerSpawnData> players;
};
```

レベルデータローダー

レベルローダー側で、自キャラの出現データを一塊で扱うために構造体を定義する。

今回は特殊なデータはないので、普通に座標と回転のみでよい。

ゲーム側の対応

実行結果

日本工学院専門学校
デザインカレッジ

```
{
  "type": "PlayerSpawn",
  "name": "PlayerSpawn",
  "transform": {
    "translation": [
      0.0,
      -19.381359100341797,
      0.0
    ],
    "rotation": [
      0.0,
      -0.0,
      0.0
    ],
    "scaling": [
      1.0,
      1.0,
      1.0
    ]
  }
}
```

"type" 欄が "PlayerSpawn" のシーンファイルが出力されるようになった。

今回は出現ポイントならではの特別なデータなどは作らなかったが、例えば複数のエリアをワープしながら進むタイプのゲームであれば、エリアの番号を出現ポイントに設定してエリアと紐づけるなどが考えられる。
(オブジェクト名で代用してもよいが。)

エクスポートの改造

日本工学院専門学校
デザインカレッジ

```
def parse_scene_recursive_json(self, data_parent, object, level):
```

export_scene.py

```
#オブジェクト種類
```

```
json_object["type"] = object.type
```

```
if "type" in object: # カスタムプロパティで指定された場合
```

```
    json_object["type"] = object["type"]
```

```
else:
```

```
    json_object["type"] = object.type
```

Jsonファイルへのエクスポート機能の改造。

といってもそれほどのことではない。

元はBlenderのオブジェクトタイプをそのままJsonに書き出していたが
["type"]カスタムプロパティがある場合はその値（名前）を書き出すように改造する。

これでオブジェクトの種類を自分で勝手に増やせるようになる。

シーンデータのエクスポート

エクスポートの改造

日本工学院専門学校
デザインカレッジ

```
def parse_scene_recursive_json(self, data_parent, object, level):
```

export_scene.py

```
#オブジェクト種類
```

```
json_object["type"] = object.type
```

```
if "type" in object: # カスタムプロパティで指定された場合
```

```
    json_object["type"] = object["type"]
```

```
else:
```

```
    json_object["type"] = object.type
```

Jsonファイルへのエクスポート機能の改造。

といってもそれほどのことではない。

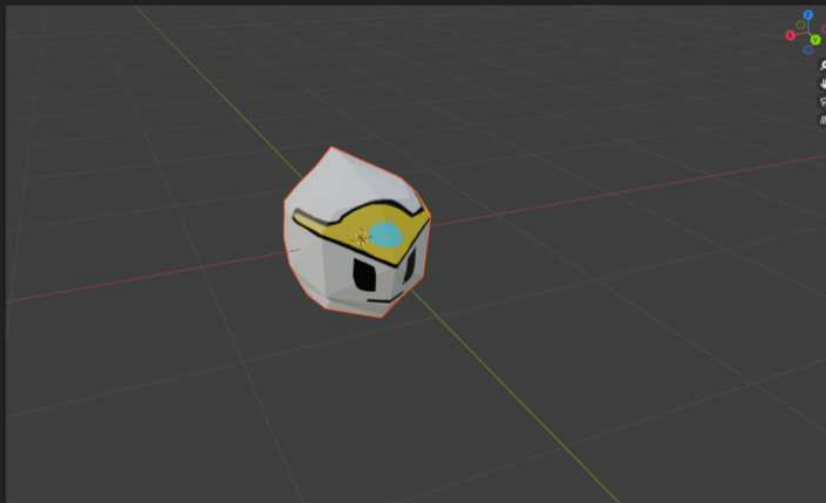
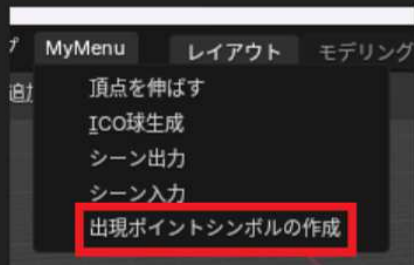
元はBlenderのオブジェクトタイプをそのままJsonに書き出していたが
["type"]カスタムプロパティがある場合はその値（名前）を書き出すように改造する。

これでオブジェクトの種類を自分で勝手に増やせるようになる。

シーンデータのエクスポート

実行結果

日本工学院専門学校
デザインカレッジ



メニューから出現シンボルを配置できるようになった。

出現シンボルは複数配置でき、モデルデータは共有しているので2体目以降の配置ではファイル読み込みが発生せずメモリにも優しい。

メニューに登録する

日本工学院専門学校
デザインカレッジ

```
my_menu.py
#トッパバーの拡張メニュー
class 自作メニュー(bpy.types.Menu):

    # サブメニューの描画
    def draw(self, context):
        #トッパバーのサブメニューにシンボル作成オペレータを追加
        自分で考えよう
```

シンボル作成（配置）オペレーターをメニューに登録する。

シンボル作成オペレーターから読み込みオペレータを実行する仕組みにしたので、読み込みオペレータの方はメニューに登録する必要はない。

メニューに登録

シンボルの配置

日本工学院専門学校
デザインカレッジ

```
print("出現ポイントのシンボルを作成します")

# Blenderでの選択を解除する
bpy.ops.object.select_all(action='DESELECT')

# 複製元の非表示オブジェクトを複製する
object = spawn_object.copy()

# 複製したオブジェクトを現在のシーンにリンク（出現させる）
bpy.context.collection.objects.link(object)

# オブジェクト名を変更
object.name = MYADDON_OT_spawn_create_symbol.object_name

return {'FINISHED'}
```

つづき

読み込み済みの非表示オブジェクトを
元データとして、複製する。

そのままでは複製したオブジェクトも
非表示状態なので、コレクション
（シーン）にリンクすることで
シーンに出現させる。

※最初に選択を解除しているのは、
これをしないと元々選択していたオブジェ
クトと複製したオブジェクトで複数選択状
態になってしまうのを防ぐため。
ピンとこなければこの処理を削って
違いを試してみるとよい。

読み込み済みモデルの検索

日本工学院専門学校
デザインカレッジ

つづき

```
def execute(self, context):  
    # 読み込み済みのコピー元オブジェクトを検索  
    spawn_object = bpy.data.objects.get(MYADDON_OT_spawn_import_symbol.prototype_object_name)  
  
    # まだ読み込んでいない場合  
    if spawn_object is None:  
        # 読み込みオペレータを実行する  
        bpy.ops.myaddon.myaddon_ot_spawn_import_symbol('EXEC_DEFAULT')  
        # 再検索。今度は見つかるはず  
        spawn_object = bpy.data.objects.get(MYADDON_OT_spawn_import_symbol.prototype_object_name)
```

複製元となるモデルデータを検索する。

読み込みオペレータを実行済みであれば検索がヒットするはずで、
未実行であれば読み込みオペレータを実装して読み込みを行い、再度検索する。

ここまでで理論上は必ずモデルデータが取得できているはず。（読み込みに失敗したのでなければ）

モジュールのimportとクラスリスト登録

```
__init__.py
# モジュールのインポート
...
+ 自分で考えよう

# Blenderに登録するクラスリスト
classes = (
+     自分で考えよう
)
```

うっかり忘れないように、
自作クラスを作成したら
すぐに `__init__.py` の方に登録しておこう。

オペレータクラス

日本工学院専門学校
デザインカレッジ

```
spawn.py
#オペレータ 出現ポイントのシンボルを作成・配置する
class 出現ポイントシンボル作成オペレータ(bpy.types.Operator):
    bl_idname = 適切に設定する
    bl_label = "出現ポイントシンボルの作成"
    bl_description = "出現ポイントのシンボルを作成します"
    bl_options = {'REGISTER', 'UNDO'}

    def execute(self, context):
        つづく
```

つづいてシンボルの作成と配置機能に
ついてもオペレータとして実装していく。

新クラスは新しくモジュール用の
スクリプトファイルを作成し、記述する。

シンボルの配置（作成）

実行結果

日本工学院専門学校
デザインカレッジ



オペレータを実行してもシーン（コレクション）には
シンボルが現れないが、モデルデータは読み込んだ状態に
なる。（見た目では確認できない）

シーンへの配置をやめる

日本工学院専門学校
デザインカレッジ

spawn.py

```
# オブジェクトの種類を設定
object["type"] =
    MYADDON_OT_spawn_import_symbol.
    object_name

+ # メモリ上にはおいておくがシーンから外す
  bpy.context.collection.objects.unlink(object)

return {'FINISHED'}
```

OBJファイルをインポートすると自動的にシーンに配置される（Blenderのメニューからインポートしたときと同じ）が、データベースに登録だけしておいて、シーンからは外すことが可能だ。（シーンから外しても前ページのオブジェクト名による検索にはヒットする）

これにより、モデルファイルの読み込みと配置を完全に分離することができる。
今度こそ、純粋なシンボル読み込みオペレータの完成である。

重複読み込み防止

日本工学院専門学校
デザインカレッジ

```
def execute(self, context):  
    print("出現ポイントのシンボルをImportします")  
  
    # 重複ロード防止  
    spawn_object = bpy.data.objects.get(MYADDON_OT_spawn_import_symbol.prototype_object_name)  
    if spawn_object is not None:  
        return {'CANCELLED'}  
  
    # スクリプトが配置されているディレクトリの名前を取得する  
    addon_directory = os.path.dirname(__file__)
```

spawn.py

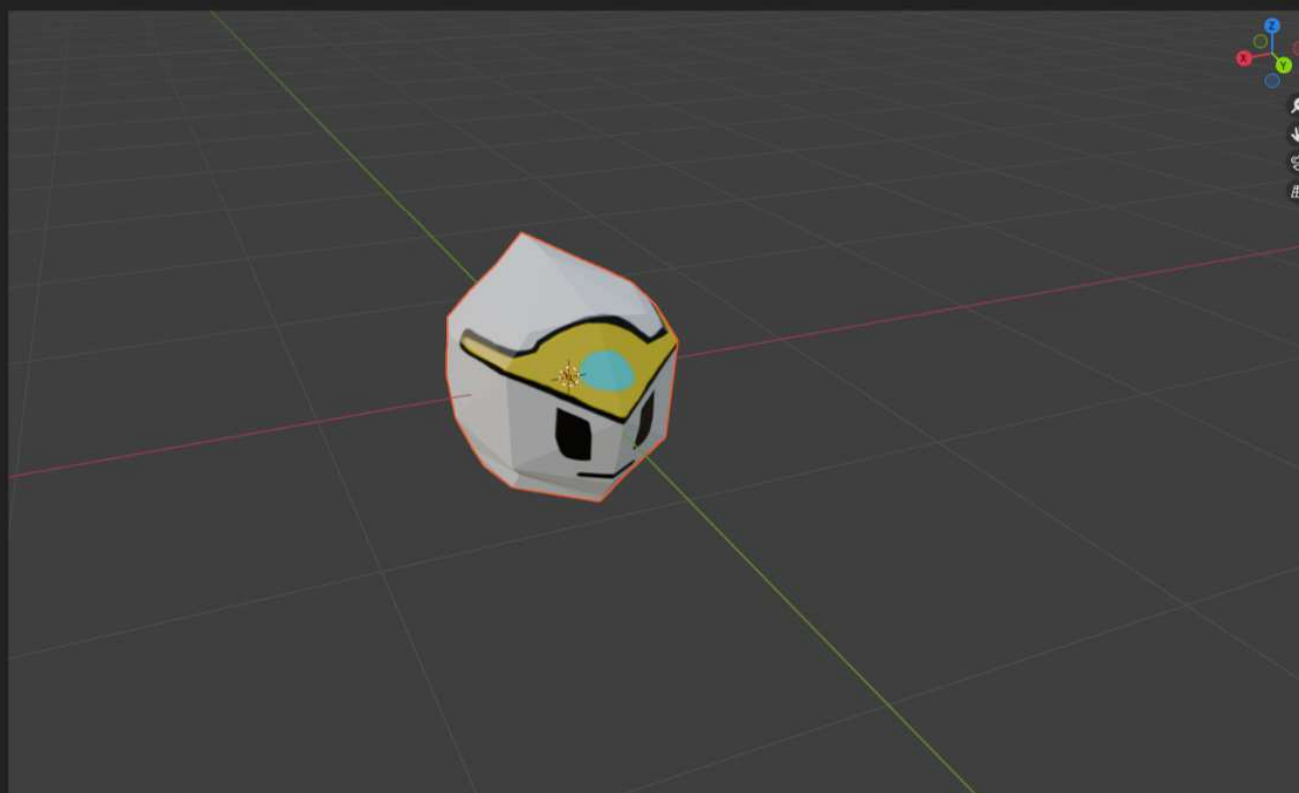
最初にも言ったが、同じ見た目のモデルを複数配置する際に
毎回モデルファイルを読み込みにいくのは処理・メモリの無駄遣いである。

そこで、Blenderのデータベースを検索して、
同じ名前のオブジェクトがあったら読み込みをスキップする。

量産の準備

実行結果

日本工学院専門学校
デザインカレッジ



オペレータなので、
【01_05.Operator】でやったよう
にメニューに登録すれば
実行確認できる。

オブジェクトの名前変更

日本工学院専門学校
デザインカレッジ

つづき

```
# アクティブなオブジェクトを取得
object = bpy.context.active_object
# オブジェクト名を変更
object.name =
    MYADDON_OT_spawn_import_symbol.
    prototype_object_name

# オブジェクトの種類を設定
object["type"] =
    MYADDON_OT_spawn_import_symbol.
    object_name

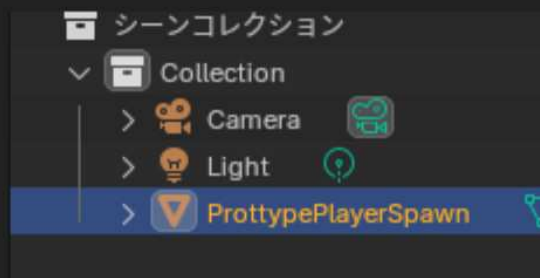
return {'FINISHED'}
```

spawn.py

```
#オペレータ 出現ポイントのシンボルを読み込む
class スポーンポイントシンボル読み込みオペレータ(省略):
    ...省略...
    prototype_object_name = "ProtttypePlayerSpawn"
    object_name = "PlayerSpawn"
```

生成したオブジェクトについて

- Blenderシーン上での名前を変更する。
- カスタムプロパティ「"type"」に専用のあらたな種類を設定する。

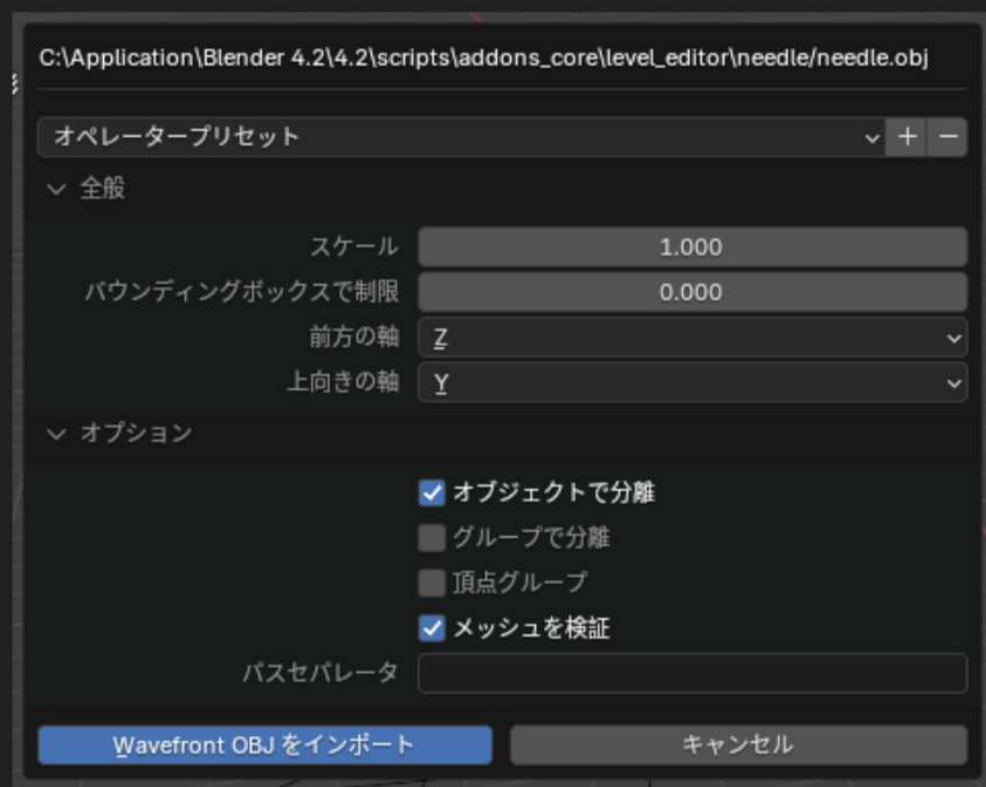


オブジェクト名は分かりやすさのためにつけている
だけなので、後から変更してもかまわない。

これで読み込みオペレータは完成。

invoke()だとどうなるの？

日本工学院専門学校
デザインカレッジ



前ページの話で '**INVOKE_DEFAULT**' を指定すると、`execute()` でなく `invoke()` が呼び出される。

基本的には `invoke()` から `execute()` が呼び出されるので最終的に目的は達成できるのだが、決まったモデルを読み込ませたいのに左図のようにいちいちダイアログが出てオプションを選択させられるのは不便すぎてどうにも勘弁願いたいので、

'**EXEC_DEFAULT**' を指定することでダイアログを回避するわけである。

モデルファイルの読み込み

日本工学院専門学校
デザインカレッジ

つづき

```
# オブジェクトをインポート
bpy.ops.wm.obj_import('EXEC_DEFAULT',
    filepath=full_path, display_type='THUMBNAIL',
    forward_axis='Z', up_axis='Y')

# 回転を適用
bpy.ops.object.transform_apply(location=False,
    rotation=True, scale=False, properties=False,
    isolate_users=False)
```

```
import bpy.ops
```

実際にモデルファイルを読み込む処理。

Blenderに標準で用意されたアドオンを使用する。

`bpy.ops.wm.obj_import()` の第一引数
'EXEC_DEFAULT'は、他のオペレータを呼び出すときに `invoke()` でなく `execute()` を実行させるためのオプションである。

`invoke()` と `execute()` の違いはオペレータによるのだが、この場合でいえば、必要な情報を引数で渡すことを条件として、ダイアログウィンドウを開かずにインポートを行えるというメリットがある。

ついでに回転の適用もこの時点で行っておく。
回転の適用については【01_12.ローダーと配置】参照

ファイルパスの設定

日本工学院専門学校
デザインカレッジ

つづき

```
def execute(self, context):  
    print("出現ポイントのシンボルをImportします")  
    # スクリプトが配置されているディレクトリの名前を取得する  
    addon_directory = os.path.dirname(__file__)  
    # ディレクトリからのモデルファイルの相対パスを記述  
    relative_path = "player/player.obj"  
    # 合成してモデルファイルのフルパスを得る  
    full_path = os.path.join(addon_directory, relative_path)
```

```
import os
```

モデルを読み込む前段階として、
モデルファイルの適切なファイルパスをフルパスで取得（合成）する。

os.pathモジュールの詳細については [pythonの公式ドキュメント](#) を参照。

モジュールのimportとクラスリスト登録

```
__init__.py
# モジュールのインポート
...
+ 自分で考えよう

# Blenderに登録するクラスリスト
classes = (
+     自分で考えよう
)
```

うっかり忘れないように、
自作クラスを作成したら
すぐに `__init__.py` の方に登録しておこう。

オペレータクラス

日本工学院専門学校
デザインカレッジ

spawn.py

```
#オペレータ 出現ポイントのシンボルを読み込む
class 出現ポイントシンボル読み込みオペレータ(bpy.types.Operator):
    bl_idname = 適切に設定する
    bl_label = "出現ポイントシンボルImport"
    bl_description = "出現ポイントのシンボルをImportします"

    def execute(self, context):
        つづく
```

新クラスは新しくモジュール用の
スクリプトファイルを作成し、記述する。

```
import 自分で考えよう
```

必要なモジュールのimportも併せて記述する。

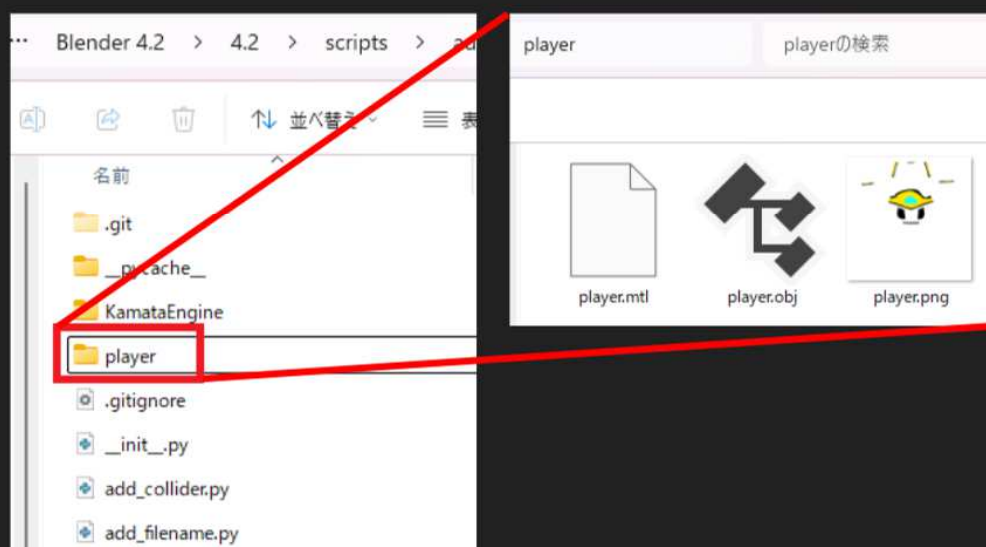
ゲーム側でもやっているはずのことだが
同じ見た目のモデルを複数配置する際に
毎回モデルファイルを読み込みにいくのは
処理・メモリの無駄遣いである。

ということでモデルデータの読み込み処理
をオペレータクラスにまとめておき、
出現ポイントを初めて設置するときだけ読
み込みが発動するような仕組みを目指す。

出現ポイント（SpawnPoint）関連の処理
は spawn.py ファイルを新規作成し、
そこにまとめていこう。

モデルファイルの配置

日本工学院専門学校
デザインカレッジ



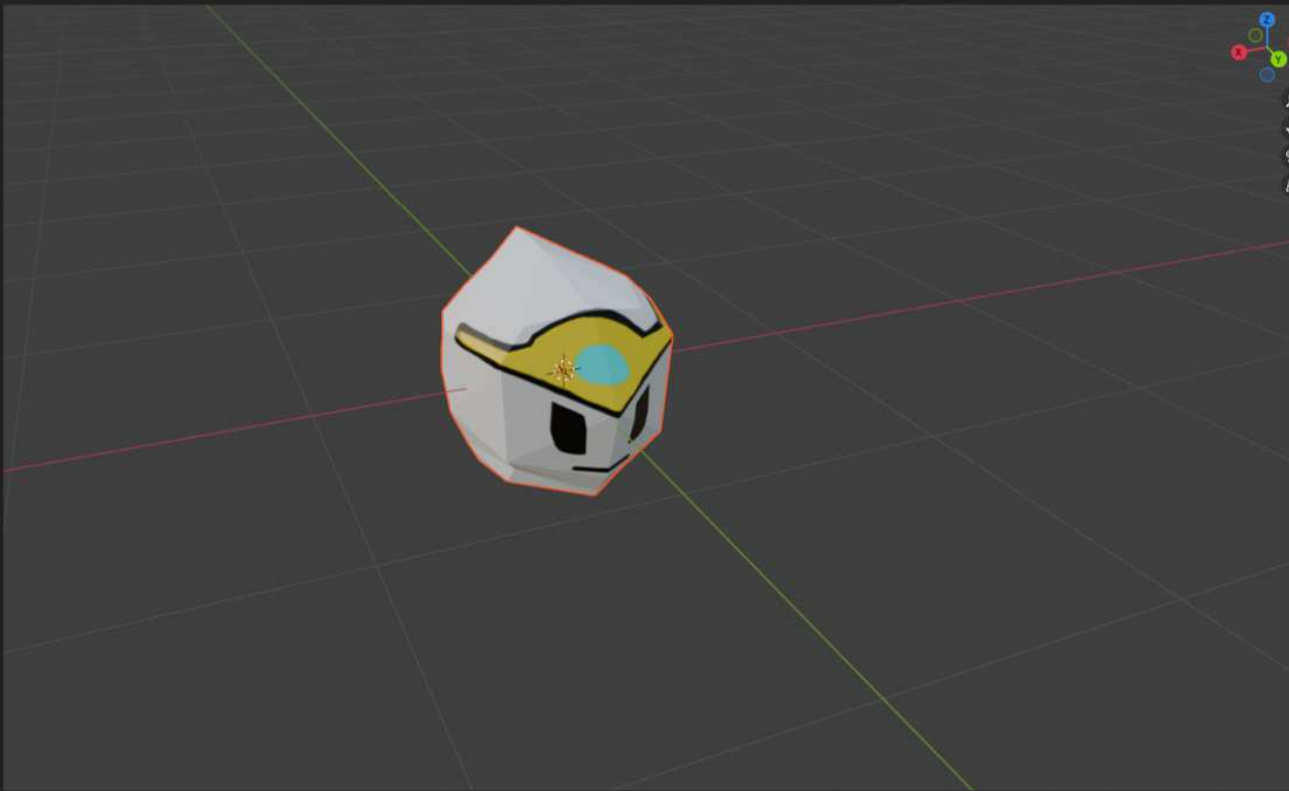
スクリプトから読み込ませるための
モデルファイルを事前に配置しておく。

今回は自キャラから実装するので、
アドオンフォルダー内の「level_editor」フォルダ
（このアドオン自体のフォルダ）に
サブフォルダを掘ってモデルファイルを
配置することにする。

モデルの種類はBlender側が対応していれば
なんでもよいのだが、今回はobjで話を進める。
glTF形式などは自分で応用してみよう。

シンボル

日本工学院専門学校
デザインカレッジ



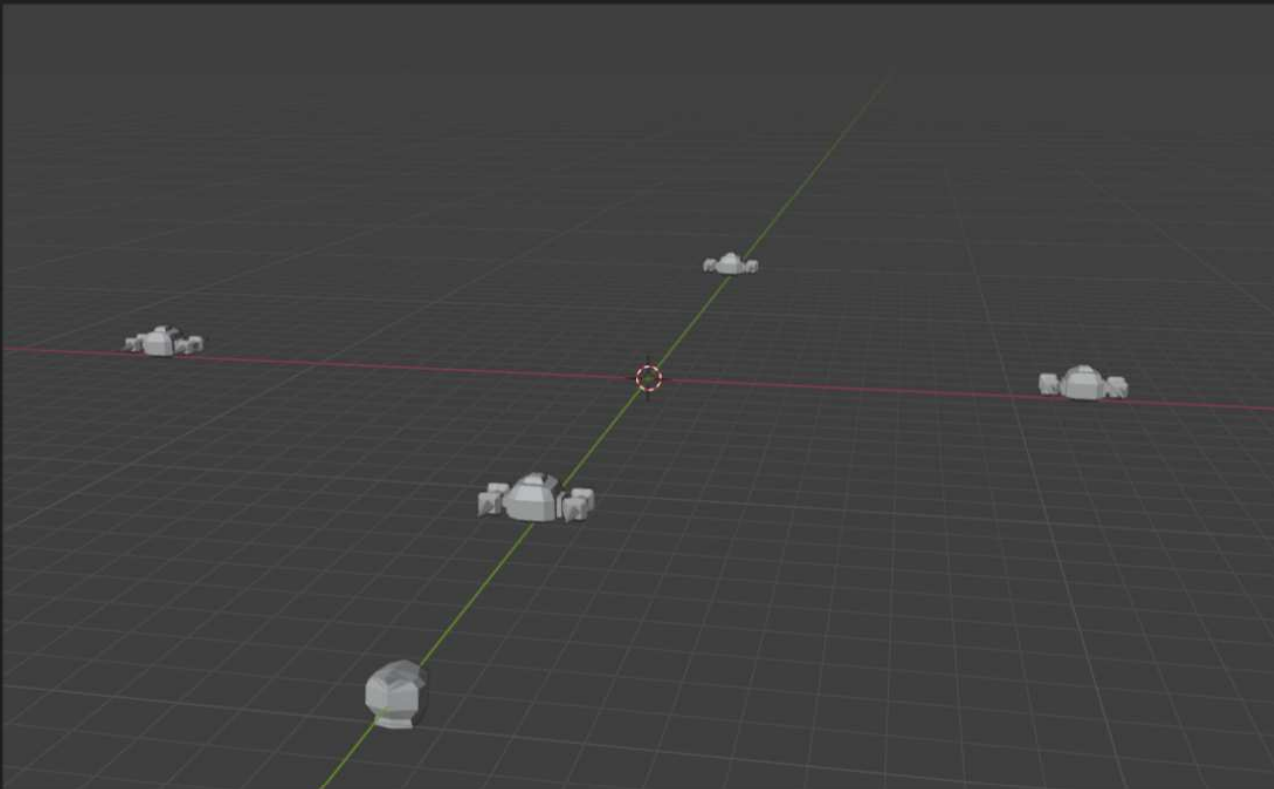
Blender上で、地形・背景に
混ざって自キャラや敵キャラの
配置ポイントを設置したいわけだ
が、Blender上でも見た目でそれと
分かるようにしておく必要がある。

デフォルトの球やボックスでは
分かりづらいので、
自キャラなら自キャラ、敵なら
敵と分かるように専用のモデルを
読み込んで設置したい。

シンボルの読み込み

今回の目標

日本工学院専門学校
デザインカレッジ



Spawn（スポン）とは
ゲーム用語で
キャラクター、敵などが
発生すること。

今回はフィールド上での
自キャラの出現位置を
Blender上でSpawnPoint
として配置できるようにする。

TL1

02_04.自キャラSpawnPoint